



Compilers

PARSING & STD

ARAIZA VALDÉS DIEGO ANTONIO
ARROYO SOLANO VICTOR JULIAN
JARAMILLO RODRÍGUEZ LESLIE CITLALLI
SALAS HERNÁNDEZ CAMILA ALEXANDRA
VELAZQUEZ CAUDILLO OSBALDO

TOKENS

- **Keywords:** int | float | for | while | if | else | return | void | char | double | long | short | unsigned | signed | struct | typedef
- **Identifier:** [A-Za-z_]\w* (includes variable names and function names)
- **Punctuation:** (,), {, }, ;, ,
- **Operator:** == | != | <= | >= | \+\+ | -- | \+= | -= | *= | /= | %= | && | \|\| | [+ \- * / % < > = ! & |]
- **Constant:** \d+(?:\.\d+)? (integers and floats)
- **Literal:** "([^\\"\\]|\\.)*" | \'([^\'\\]|\\.)*\' (strings with escapes)

STATEMENTS

- Variable declaration (with or without initialization): `int x;` or `int x = 5;`
- Function declaration/definition: `int add(int a, int b) { ... }`
- Function calls (with arguments): `printf("Hello");` or `add(3, 5);`
- Assignment: `x = 10;`
- Expressions (arithmetic, logical, relational):
`x + y`, `a && b`, `x < 5`
- Return statements: `return x;`
- Code blocks: `{ ... }`

FEATURES

- **Lexical Analysis:** Complete tokenization via `adapter_lexer.py`
- **Two-pass Function Analysis:** First pass collects declarations, second pass validates calls
- **Syntax Analysis:** LL(1) recursive descent parser
- **Semantic Analysis:** Type checking, symbol validation, and function argument validation
- **Error Reporting:** Clear error messages indicating the type of error
- **Builtin Functions:** Special handling for `printf()` and `scanf()`
- **Type System:** Support for `int`, `float`, `string`, `bool` with implicit conversions

IMPLEMENTATION

- **main.py:** Core implementation of the parser, AST construction, and semantic analyzer.
- **parser.py:** Runner / CLI wrapper that invokes `run(src: str)` from `main.py` (entry point for scripts or command-line use).
- **lexer/adaptlex.py:** Adapter that normalizes tokens to the parser's expected format, using `user_lexer` (which wraps `tokenize_std`) as its main interface.

NODE TYPES AST

Var

Program

ExprStmt

Return

Decl

Num

FuncDecl

String

Block

BinOp

Assign

FuncCall

UnaryOp

RESULTS

Success

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers\08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//FLOAT Y STRING
float calcular(float a, float b) {
    float resultado = a * b;
    return resultado;
}

int main() {
    string mensaje = "Hola Mundo";
    float pi = 3.14159;
    float r = calcular (5.5, 2.0);
    printf (r);
    return 0;
}
^Z
Parsing Success!
SDT Verified!
```

RESULTS

Success

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers\08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//BOOLEANOS, COMPARACIONES Y OPERADORES
int main() {
    int x = 10;
    int y = 20;
    bool mayor = x > y;
    bool igual = x == y;
    bool resultado = mayor && igual;
    bool otro = mayor || igual;
    return 0;
}
^Z
Parsing Success!
SDT Verified!
```


RESULTS

Success

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers/08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//CONCATENACION
int main() {
    string nombre = "Juan";
    string apellido = "Perez";
    string completo = nombre + apellido;
    return 0;
}
^Z
Parsing Success!
SDT Verified!
```

RESULTS

Semantic Error

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers\
s/08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//VARIABLE NO DECLARADA
int main() {
    x = 10;
    return 0;
}
^Z
Parsing Success!
SDT error...
Variable 'x' no declarada
```

RESULTS

Semantic Error

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers
s/08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//OPERACION INCOMPATIBLE
int main() {
    int x = 10;
    string y = "hola";
    int z = x + y;
    return z;
}
^Z
Parsing Success!
SDT error...
Operación '+' incompatible entre int y string
```

RESULTS

Semantic Error

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers\g5\08\parser.py"
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//FUNCION NO DECLARADA
int main() {
    saludar();
    return 0;
}
^Z
Parsing Success!
SDT error...
Función 'saludar' no declarada
```

RESULTS

Semantic Error

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers/08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//OPERADOR LOGICO CON NO BOOLEANO
int main() {
    int x = 10;
    int y = 20;
    bool resultado = x && y;
    return 0;
}
^Z
Parsing Success!
SDT error...
Operación lógica '&&' espera booleanos, obtuvo int y int
```


RESULTS

Parsing Error

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers/08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//PARENTESIS SIN CERRAR
int main() {
    int x = (10 + 5;
    return x;
}
^Z
Parsing error...
Se esperaba ')' (got SEMI ';')
```

RESULTS

Parsing Error

```
PS C:\Users\Diego\OneDrive\Documentos\GIT\Compilers_Team8\Parser&STD\unam\fi\compilers
s/08/parser.py
Introduce el código fuente (finaliza con Ctrl+Z y Enter en Windows, Ctrl+D en Unix):
//FALTA PUNTO Y COMA
int main() {
    int x = 10
    return x;
}
^Z
Parsing error...
Falta ';' al final de la declaración (got ID 'return')
```